

Ball Machine

Madina ha inventado una máquina de bolas para entretener a los participantes de la IOI. La estructura interna de la máquina es la siguiente:

- La máquina consta de N **nodos**, numerados desde 0 hasta $N - 1$. El nodo $N - 1$ se llama **raíz**.
- Para cada nodo u con $0 \leq u < N - 1$, el **padre** del nodo u es un nodo $P[u]$ con $P[u] > u$, y el nodo u es un **hijo** de $P[u]$. La raíz no tiene padre. Un nodo sin hijos se denomina **hoja**.

No conoces N ni el padre $P[u]$ de ningún nodo u . En cambio, Madina te indica M , el número de hojas, y que los índices de las hojas son $0, 1, \dots, M - 1$. Tu tarea consiste en determinar la estructura interna de la máquina haciendo operaciones con ella.

Cada nodo de la máquina puede contener como máximo una **bola**, y cada bola tiene un **valor** que es un número entero no negativo. Decimos que un nodo tiene valor x si contiene una bola con valor x . Un nodo está **ocupado** si contiene una bola; de lo contrario, está **libre**. Inicialmente, todos los nodos están libres.

Puedes realizar dos tipos de operaciones:

- **insert** (U, X) : Intenta insertar una nueva bola con valor X en la hoja U .
 - Si la hoja U está ocupada, la operación devuelve `false` sin insertar la bola.
 - Si la hoja U está libre, la bola se coloca en el nodo U . Luego, la bola se mueve repetidamente hacia el padre de su nodo actual mientras ese padre esté libre. El movimiento se detiene cuando la bola llega a la raíz o a un nodo cuyo padre está ocupado. En este caso la operación devuelve `true`.
- **collect**(**)**: si la raíz está libre (lo que significa que la máquina está vacía), `collect` devuelve un arreglo vacío. De lo contrario, el procedimiento recursivo `traverse` descrito a continuación recopila los valores de todas las bolas que se encuentran actualmente en la máquina en un arreglo S . Inicialmente, el arreglo S está vacío, y se invoca `traverse(N - 1)`.

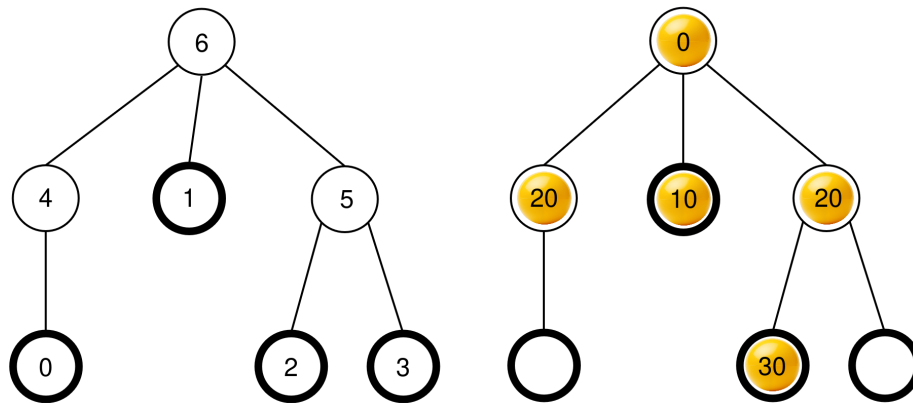
```

traverse(u):
    inserta el valor de la bola del nodo u al final de S
    sea c[u] la lista de los hijos ocupados de u
    ordena c[u] en orden no decreciente de acuerdo a los valores de las bolas
        en esos nodos. (si múltiples nodos tienen el mismo valor,
            pueden aparecer en cualquier orden)
    para cada v en c[u]:
        traverse(v)

```

Cuando este procedimiento termina, **todas las bolas son removidas** de la máquina, y `collect` retorna S .

Por ejemplo, considera una máquina con $N = 7$ nodos y arreglo de padres $P = [4, 6, 5, 5, 6, 6]$. La imagen de la izquierda muestra la máquina vacía con los índices de los nodos, mientras la imagen a la derecha muestra una configuración posible de bolas después de algunas operaciones de `insert` (la secuencia de operaciones se muestra en la sección de Ejemplos).



Cuando se llama la función `collect()`, la raíz (nodo 6) está ocupada, entonces S se inicializa como $S = []$ y se invoca `traverse(6)`.

traverse(6): el nodo 6 tiene valor 0; insertando ese valor en S deja $S = [0]$. Los hijos del nodo 6 son 4, 1, y 5, que están todos ocupados. Sus valores son 20, 10, y 20, respectivamente. Ordenando en orden no-decreciente nos deja $c[6] = [1, 5, 4]$ (nota que $c[6] = [1, 4, 5]$ podría ser válido, dado que los nodos 4 y 5 tienen el mismo valor). El procedimiento invoca recursivamente a `traverse` en cada nodo del arreglo $c[6]$ en ese orden.

- **traverse(1):** el nodo 1 tiene valor 10; al agregarlo a S queda $S = [0, 10]$. El nodo 1 no tiene hijos ocupados, se termina la llamada.
- **traverse(5):** el nodo 5 tiene valor 20; al agregarlo a S queda $S = [0, 10, 20]$. El nodo 5 tiene un hijo ocupado, el 2, luego $c[5] = [2]$ y se invoca recursivamente al procedimiento `traverse(2)`.

- **traverse(2)**: el nodo 2 tiene valor 30; al agregarlo a S queda $S = [0, 10, 20, 30]$. El nodo 2 no tiene hijos ocupados, se termina la llamada.
- La ejecución retorna a **traverse(5)** que ha procesado todos los hijos de $c[5]$, entonces termina la llamada.
- **traverse(4)**: el nodo 4 tiene valor 20; al agregarlo a S queda $S = [0, 10, 20, 30, 20]$. El nodo 4 no tiene hijos ocupados, entonces termina la llamada.

Todas las llamadas se completaron y no hay más nodos para procesar. Finalmente, cada bola es removida de la máquina y **collect** retorna el arreglo $S = [0, 10, 20, 30, 20]$.

Tu tarea es determinar el número de nodos N y reportar un arreglo de padres $R = [R[0], R[1], \dots, R[N-2]]$ que describa la estructura de la máquina, pero con un etiquetado potencialmente distinto de los nodos que no son hojas ni la raíz. Formalmente, un arreglo reportado R es considerado correcto si es posible asignar una 'única etiqueta $L[u]$ con $0 \leq L[u] < N$ a cada nodo u de la máquina, tal que:

- $L[u] = u$ para todo $0 \leq u < M$ y $u = N-1$, y
- $L[P[u]] = R[L[u]]$ para todo $0 \leq u < N-1$.

No es requerido que $R[u] > u$. La máquina **debe quedar vacía** cuando reportes tu solución.

Sea K el número de operaciones **collect** realizadas, y B el valor máximo escrito en cada bola a lo largo de todas las llamadas de **insert**. Sea $C = K + B$. C **no debe exceder** 1000. Tu score en algunas subtareas depende del valor de C .

Detalles de implementación

Debes implementar el siguiente procedimiento:

```
std::vector<int> find_structure(int M)
```

donde:

- M : El número de hojas en la máquina.
- El procedimiento es llamado exactamente una sola vez para cada caso de prueba.
- El procedimiento debe retornar un arreglo $R = [R[0], R[1], \dots, R[N-2]]$ de longitud $N-1$, un arreglo correcto de padres.

Para interactuar con la máquina, tu procedimiento puede llamar a los siguientes dos procedimientos:

El primero es:

```
bool insert(int U, int X)
```

- U : El índice de la hoja donde la bola debe ser insertada. Se debe cumplir la condición $0 \leq U < M$.
- X : El valor entero de la bola. Se debe cumplir la condición $0 \leq X \leq 1000$.
- Este procedimiento retorna `true` si la bola fue insertada correctamente, o `false` si la hoja U estaba ocupada.
- Este procedimiento puede ser llamado un máximo de 500 000 veces.

El segundo procedimiento es:

```
std::vector<int> collect()
```

- Recolecta los valores de todas las bolas insertadas, vacía la máquina, y retorna el arreglo S .

Si durante algún punto de la ejecución de tu programa el valor de C excede 1000, tu solución recibirá el veredicto: `Output isn't correct: Too many resources used`.

La estructura de la máquina es establecida antes de que el procedimiento `find_structure` sea llamado. El evaluador es determinista en el sentido de que si se ejecuta dos veces y ambas ejecuciones realizan la misma secuencia de operaciones, las llamadas a la función `collect` devolverán los mismos arreglos.

Restricciones

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtareas

Subtarea	Score	Restricciones adicionales
1	5	La raíz tiene exactamente M hijos.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	No hay restricciones adicionales.

En las sub-tareas 3 y 4, tu score depende del valor de C como se explica a continuación.

SubTarea 3 (25 puntos)

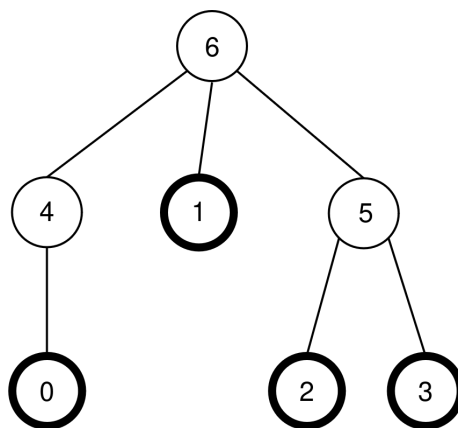
Límites	Score
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtarea 4 (60 puntos)

Limites	Score
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Ejemplo

Considera la misma máquina que se usa en la descripción del problema, con $N = 7$, $M = 4$, y $P = [4, 6, 5, 5, 6, 6]$. La máquina se muestra de nuevo a continuación:



El evaluador hace la siguiente llamada:

```
find_structure(4)
```

Entonces el procedimiento puede hacer la siguiente secuencia de llamadas:

1. **insert(0, 0)**: la hoja 0 está libre, entonces una bola con valor 0 se pone en la hoja 0. El nodo $P[0] = 4$ está libre, entonces la bola se mueve a la posición 4. El nodo $P[4] = 6$ está libre,

entonces la bola se mueve al nodo 6. Como el nodo 6 es la raíz, se termina la llamada. El procedimiento retorna `true`.

2. `insert(3, 20)`: la hoja 3 está libre, por lo tanto una bola con valor 20 se coloca en la hoja 3. El nodo $P[3] = 5$ está libre, así que la bola se mueve al nodo 5. Como $P[5] = 6$ está ocupada, el movimiento termina. La llamada retorna `true`.

3. `insert(1, 10)`: La hoja 1 está libre, entonces una bola con valor 10 se pone en la hoja 1. Como $P[1] = 6$ está ocupada, la bola se queda en la casilla 1. La llamada retorna `true`.

4. `insert(2, 30)`: La hoja 2 está libre, así que una bola con valor 30 es colocada en la hoja 2. Como $P[2] = 5$ está ocupada, la bola permanece en la casilla 2. La llamada retorna `true`.

5. `insert(1, 25)`: la hoja 1 está ocupada, así que la bola no se coloca en la hoja 1. y la llamada retorna `false`.

6. `insert(0, 20)`: La hoja 0 está libre, así que la bola con el valor 20 se pone en la hoja 0. El nodo $P[0] = 4$ está libre, entonces la bola se mueve al nodo 4. Como $P[4] = 6$ está ocupada, el movimiento termina. La llamada retorna `true`.

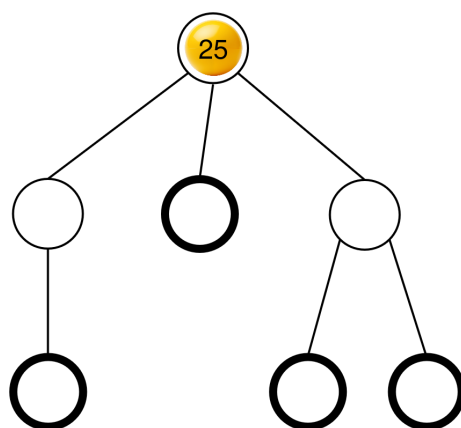
La configuración resultante de bolas ya fue mostrada en la descripción de la tarea.

A continuación, el procedimiento puede invocar:

```
collect()
```

La ejecución de esta llamada ya fue explicada en la descripción de la tarea, y su llamada retorna $S = [0, 10, 20, 30, 20]$. Al final, la máquina queda vacía de nuevo.

Luego, el procedimiento puede llamar `insert(2, 25)`. La hoja 2 está libre, entonces una bola con valor 25 se pone en la hoja 2. Esta bola se mueve entonces al nodo 5, y luego al nodo 6, donde se detiene. La llamada retorna `true`. La configuración resultante de bolas se muestra en la siguiente Figura:



Finalmente, el procedimiento puede llamar a `collect()` que retorna el arreglo $S = [25]$ y vacía la máquina.

Hay dos arreglos de padres correctos que `find_structure` puede devolver: $R = P = [4, 6, 5, 5, 6, 6]$ (que corresponde con las etiquetas $L = [0, 1, 2, 3, 4, 5, 6]$), y $R = [5, 6, 4, 4, 6, 6]$ (que corresponde al

etiquetado $L = [0, 1, 2, 3, 5, 4, 6]$). En este ejemplo, $K = 2$ y $B = 30$, así que $C = 32$.

Ejemplo de Evaluador

Formato de Entrada:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formato de Salida:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Aquí, Q es la longitud del arreglo R retornado por `find_structure`.